

Design Rationale

REQ4: The Wayward Seller of Wares

A. Merchant Implementation

Design 1: Direct Actor/ConversationalActor Extension

Pros	Cons
Simpler class hierarchy with fewer levels of abstraction	Less separation between merchant functionality and actor functionality
Direct access to Actor methods without intermediaries	More difficult to share merchant-specific behaviour across actors

Design 2: Merchant Interface Implementation

Pros	Cons
Clear separation of concerns between actor and merchant functionality	Slightly more complex with added interface layer
Any actor type can become a merchant via interface implementation	Requires understanding of interface implementation
Promotes loose coupling between actor and merchant roles	
Easier to extend with new merchant types or features	

B. Item Effects Management

Design 1: Direct Effect Implementation in Weapon Classes

Pros	Cons
Simple implementation with effects in weapons	Code duplication across similar weapons
No additional abstractions to manage	Difficult to reuse effects across items
Direct control over each weapon's effects	Changing shared effects needs multiple class updates

Design 2: Separate Effect Classes with Composition

Pros	Cons
Promotes code reuse via composition	More complex architecture with additional classes
Effect logic encapsulated in dedicated classes	Requires understanding of composition pattern
Easy to add or combine effects	
Allows runtime effect composition	

C. Merchant-Item Relationship

Design 1: Direct Item-to-Merchant Binding

Pros	Cons
Simple implementation with direct relationships	Tight coupling between merchants and inventory
No extra abstractions needed	Cannot vary item price/effect per merchant easily
Easy to understand merchant-item linkage	Modifying relationships requires code changes

Design 2: Offer-based System

Pros	Cons
Decouples merchants from item implementation	More complex due to added abstraction
Supports different prices/effects per merchant	
Offers can be modified dynamically	
Adheres to Open-Closed Principle	

D. Weapon Sold Effects Implementation

Design 1: Sellable Interface with a dedicated soldEffects method

Pros	Cons
Defines contract for sellable items	Adds an interface layer to the system
Enables polymorphism for saleable items	Requires implementation on all sellable items
Respects SRP by isolating sale behaviour	
Reduces duplication via centralized effect logic	

Design 2: Direct Effect Implementation in Merchant Classes

Pros	Cons
Simple design without interfaces	Tight coupling between merchants and effects
Direct effect control within merchant class	Effect logic duplicated across merchants
	Violates DRY principle with repeated logic

Final Design Choice

The final design utilizes Design 2 for Merchant Implementation, Design 2 for Item Effects Management, Design 2 (Offer-based System) for Merchant-Item Relationship and Design 1 (Sellable Interface) for Weapon Sold Effects Implementation

The system design adheres strongly to the Single Responsibility Principle (SRP), ensuring that each component has a clear and focused purpose. The Merchant interface is responsible solely for merchant-specific capabilities, the PurchaseEffect interface handles effect application, the Offer class encapsulates trade details, and the Sellable interface defines item sale behaviour. This separation avoids overloading classes with multiple concerns.

The Open-Closed Principle (OCP) is followed by making the system extensible without requiring modification of existing code. We can introduce new merchants by implementing the Merchant interface, create new effects via the PurchaseEffect interface, and add new sellable items through the Sellable interface. As an example, integrating a new effect like "SpawnSpiritGoatEffect" simply requires a new class that implements PurchaseEffect, no existing classes need alteration. This modularity enables the system to evolve organically.

The Liskov Substitution Principle (LSP) is also respected. Implementations such as MerchantKale and Sellen follow the Merchant interface, all effect classes comply with the PurchaseEffect interface, and sellable weapons correctly implement the Sellable interface. This ensures that any class using these interfaces can rely on consistent, expected behaviour regardless of specific implementations.

With the Interface Segregation Principle (ISP), the system avoids large, catch-all interfaces. For instance, the Merchant interface has only a method called addOffers() in which allows Actors to add new Offers for selling different items with new or existing effects.

The Dependency Inversion Principle (DIP) is applied by depending on abstractions rather than concrete implementations. Merchants interact with the Offer abstraction

rather than directly with item classes, and purchase processes depend on the PurchaseEffect interface instead of specific effects. For example, MerchantKale does not need to know the inner workings of the BroadSword or DragonSlayerGreatSword classes, but simply utilizes the Offer abstraction, enhancing flexibility and reducing coupling.

The system also follows the DRY (Don't Repeat Yourself) principle through centralized, reusable effect classes. Common functionalities, such as healing or stat modifications, are implemented once and reused wherever needed. Without this design, such logic would need to be duplicated across various weapon or merchant classes. The HealEffect class, for instance, can be used by both the Broadsword and the Katana without reimplementing the healing logic.

Connascence is minimized through clean interfaces and limited interdependencies. Purchase effects are agnostic to the internals of actors or merchants and rely solely on public interfaces. Weapons remain unaware of who sells them or at what price. Likewise, merchants do not require knowledge of how the effects in their offers are implemented. This decoupling reduces the risk of cascading failures when one part of the system changes.

In terms of future maintainability, the design is highly adaptable. Adding a new weapon requires only implementing the Sellable interface. Creating a new effect involves implementing the PurchaseEffect interface. If merchant behavior changes, only the specific Merchant class needs updating. For instance, if a future feature demands weapons that apply status effects upon purchase, a new ApplyStatusEffect class can be created and integrated without modifying any existing code. This new effect can be seamlessly composed with existing ones in various offers.

The decision to adopt an offer-based system with separate effect classes over a direct merchant-item binding model was made for its superior flexibility and extensibility. In a direct binding model, adding a merchant who sells existing items with unique effects would necessitate duplicating item logic. The offer-based design allows the same item to be sold by different merchants with distinct effects and prices by simply creating new Offer instances. This not only streamlines future development but also meets the key requirement that items like the Broadsword can be sold differently by Sellen and Kale, adapting easily to new gameplay needs.